Course Project Report

Assembly-Level Execution on Integrated RISC-V Processor

EE/CE 321L/371L – Computer Architecture Lab

Name: _____ ID: _____ Section: _____

March 31, 2026

1 Part A: Base Assembly Program Execution

1.1 Objective

Execute the RISC-V assembly program developed in Lab 10 (FSM-based countdown using switches and LEDs) on the integrated single-cycle RISC-V processor from Lab 11.

1.2 Program Description

The Lab 10 program implements a Finite State Machine with two states:

- **WAIT state:** The processor continuously reads the switch value from the memory-mapped switch address (768 = `0x300`). If the value is zero, it loops. When a non-zero value is detected, it transitions to the COUNT state.
- **COUNT state:** The captured value is passed to a countdown subroutine. The subroutine displays the current count on the LEDs (address 512 = `0x200`), decrements by 1, runs a small delay loop, and repeats until the count reaches zero. It then clears the LEDs and returns to the WAIT state.

The subroutine uses proper stack conventions — it saves the return address (`ra`) and the argument register (`a2`) onto the stack before execution, and restores them before returning via `jalr`.

1.3 Assembly Listing

Listing 1: Lab 10 FSM Program

```
# Initialization
addi x28, x0, 5       # test value
addi x2, x0, 511      # sp = 511
addi x5, x0, 768      # switch address
addi x6, x0, 512      # LED address
sw   x28, 0(x5)        # write test value to switches
sw   x0, 0(x6)         # clear LEDs

# WAIT LOOP
wait: lw x11, 0(x5)    # read switch value
      beq x11, x0, wait # if zero, keep waiting
```

```
12
13 # Call countdown subroutine
14 add  x10, x11, x0      # a0 = switch value
15 jal  x1, countdown      # call subroutine
16 beq  x0, x0, clear      # after return, jump to clear
17
18 # COUNTDOWN SUBROUTINE
19 countdown:
20     addi x2, x2, -8     # push stack
21     sw   x1, 4(x2)      # save ra
22     sw   x12, 0(x2)     # save a2
23     add  x12, x10, x0   # a2 = argument
24 loop:
25     sw   x12, 0(x6)     # display on LEDs
26     beq  x12, x0, done  # if zero, done
27     addi x12, x12, -1   # decrement
28     addi x13, x0, 3     # delay counter
29 delay:
30     addi x13, x13, -1   # delay--
31     bne  x13, x0, delay # delay loop
32     beq  x0, x0, loop   # back to display
33 done:
34     sw   x0, 0(x6)      # clear LEDs
35     lw   x12, 0(x2)     # restore a2
36     lw   x1, 4(x2)      # restore ra
37     addi x2, x2, 8      # pop stack
38     jalr x0, 0(x1)      # return
39
40 # HALT
41 halt: jal x0, halt      # infinite loop
```

1.4 Instructions Used

`ADDI`, `SW`, `LW`, `ADD`, `BEQ`, `BNE`, `JAL`, `JALR`

1.5 Simulation Results

The simulation waveform confirms correct behavior:

- With switches = 3: LEDs display `0003` → `0002` → `0001` → `0000`
- With switches = 5: LEDs display `0005` → `0004` → `0003` → `0002` → `0001` → `0000`
- The processor returns to the WAIT state after each countdown completes.

(Attach simulation waveform screenshot here)

2 Part B: Instruction Extension

2.1 Objective

Implement three new RISC-V instructions of different types that are not already supported by the processor.

2.2 Selected Instructions

Table 1: New Instructions Implemented

Instruction	Type	Opcode	Description
SLT	R-type	0110011	Set Less Than (signed comparison)
BGE	B-type	1100011	Branch if Greater or Equal (signed)
LUI	U-type	0110111	Load Upper Immediate

2.3 Instruction 1: SLT (R-type)

2.3.1 Functionality

`SLT rd, rs1, rs2` sets `rd = 1` if `rs1 < rs2` (signed comparison), otherwise `rd = 0`. The encoding uses funct3 = `010` and funct7 = `0000000`.

2.3.2 Hardware Modifications

ALU.v — Added one new case for the SLT operation:

Listing 2: ALU modification for SLT

```
// Added inside the case(ALUControl) block:
4'b0111: ALUResult = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0;
```

ALUControl.v — Added routing for funct3 = `010` under ALUOp = `10`:

Listing 3: ALUControl modification for SLT

```
// Added inside the ALUOp 2'b10 case:
3'b010 : ALUControl = 4'b0111; // SLT
```

2.3.3 Datapath Flow

The instruction flows through the standard R-type path: the opcode `0110011` sets ALUOp = `10` and RegWrite = 1. The ALUControl decodes funct3 = `010` to generate ALUControl = `0111`, which triggers the signed comparison in the ALU. The result (0 or 1) is written back to the destination register.

2.4 Instruction 2: BGE (B-type)

2.4.1 Functionality

`BGE rs1, rs2, offset` branches to `PC + offset` if `rs1` $\geq$ `rs2` (signed). The encoding uses funct3 = `101` with the branch opcode `1100011`.

2.4.2 Hardware Modifications

TopLevelProcessor.v — Extended the branch logic to include BGE:

Listing 4: Branch logic modification for BGE

```
wire branchTaken = Branch & (
    (funct3 == 3'b000 & Zero)    |         // BEQ
    (funct3 == 3'b001 & ~Zero)   |         // BNE
    (funct3 == 3'b101 & ~ALUResult[31])   // BGE (NEW)
);
```

2.4.3 Datapath Flow

BGE shares the branch opcode, so MainControl sets Branch = 1 and ALUOp = `01`. The ALUControl outputs SUB (`0001`), so the ALU computes `rs1 - rs2`. If the result's sign bit (bit 31) is 0, the subtraction is non-negative, meaning `rs1` $\geq$ `rs2`. The condition `~ALUResult[31]` captures this, and the branch is taken.

2.5 Instruction 3: LUI (U-type)

2.5.1 Functionality

`LUI rd, imm` loads a 20-bit immediate into the upper 20 bits of `rd`, with the lower 12 bits set to zero. The result is `rd = imm` $\ll$ `12`.

2.5.2 Hardware Modifications

MainControl.v — Added `LUI` output signal and a new case:

Listing 5: MainControl modification for LUI

```
// New output signal added:
output reg LUI

// New case in opcode decoder:
7'b0110111: begin // LUI
    RegWrite = 1'b1;
    LUI = 1'b1;
end
```

TopLevelProcessor.v — Added LUI to the write-back multiplexer:

Listing 6: Write-back modification for LUI

```
assign regWriteData = (Jump | JumpReg) ? PCplus4
                    : (LUI ? imm : writeBackData);
```

2.5.3 Datapath Flow

LUI uses a unique opcode `0110111`. The MainControl sets RegWrite = 1 and LUI = 1. The immediate generator (immGen) already supports this opcode and produces {`instruction[31:12], 12'b0`}. The LUI signal routes this immediate value directly to the register write-back path, bypassing both the ALU and the memory. The immediate is written directly into the destination register.

2.6 Verification Test Program

A test program was written to verify all three instructions:

Listing 7: Test program for new instructions

```
addi x6, x0, 512      # LED address

# TEST 1: SLT
addi x7, x0, 5        # x7 = 5
addi x8, x0, 10       # x8 = 10
slt  x9, x7, x8       # x9 = (5 < 10) = 1
sw   x9, 0(x6)         # LEDs show 0x0001

slt  x9, x8, x7       # x9 = (10 < 5) = 0
sw   x9, 0(x6)         # LEDs show 0x0000

# TEST 2: BGE
addi x7, x0, 10       # x7 = 10
addi x8, x0, 5        # x8 = 5
addi x9, x0, 0xAA     # x9 = 170
bge  x7, x8, skip     # 10 >= 5, branch (skip next)
addi x9, x0, 0xFF     # SKIPPED if BGE works
skip: sw x9, 0(x6)    # LEDs show 0x00AA (not 0xFF)

# TEST 3: LUI
lui  x9, 0xABCDE      # x9 = 0xABCDE000
sw   x9, 0(x6)         # LEDs show 0xE000 (lower 16 bits)

halt: jal x0, halt     # infinite loop
```

2.7 Simulation Results

The simulation confirms correct operation of all three instructions:

Table 2: Part B Test Results

Test	Operation	Expected LEDs	Observed LEDs
SLT (true)	`slt x9, 5, 10`	0x0001	0x0001
SLT (false)	`slt x9, 10, 5`	0x0000	0x0000
BGE	`bge 10, 5, skip`	0x00AA	0x00AA
LUI	`lui x9, 0xABCDE`	0xE000	0xE000

(Attach simulation waveform screenshot here)

3 Part C: Program Demonstration – Fibonacci Series

3.1 Objective

Demonstrate a meaningful assembly program executing on the processor with correct stack usage for procedure calls.

3.2 Program Description

The program computes the first 10 Fibonacci numbers (1, 1, 2, 3, 5, 8, 13, 21, 34, 55) and displays each on the LEDs. It consists of two parts:

- **Main loop (PC 0–52):** Initializes registers (`prev=0`, `curr=1`, `count=10`), then iterates: calls the display subroutine, computes the next Fibonacci number (`next = prev + curr`), shifts values, decrements the counter, and loops.
- **Display subroutine (PC 72–112):** A proper function that uses the stack to save and restore `ra` (return address) and `a0` (argument). It writes the Fibonacci number to the LEDs, runs a delay loop, then restores registers and returns.

3.3 Register Usage

Table 3: Register Allocation for Fibonacci Program

Register	Purpose	Convention
x2 (sp)	Stack pointer (initialized to 511)	Callee-saved
x6 (t1)	LED address (512)	Temporary
x7 (t2)	Previous Fibonacci number	Temporary
x8 (s0)	Current Fibonacci number	Temporary
x9 (s1)	Next Fibonacci number (temp)	Temporary
x10 (a0)	Argument to subroutine	Argument
x13 (a3)	Delay counter	Temporary
x14 (a4)	Loop counter (10 iterations)	Temporary
x1 (ra)	Return address	Saved on stack

3.4 Assembly Listing

Listing 8: Fibonacci Program

```
# === INITIALIZATION ===
addi x2, x0, 511      # sp = 511 (top of data memory)
addi x6, x0, 512      # x6 = LED address
addi x7, x0, 0        # prev = 0
addi x8, x0, 1        # curr = 1
addi x14, x0, 10      # count = 10

# === MAIN LOOP ===
loop:
    add  x10, x8, x0   # a0 = curr (argument for subroutine)
    jal  x1, display    # call display_and_delay
    add  x9, x7, x8    # next = prev + curr
    add  x7, x8, x0    # prev = curr
    add  x8, x9, x0    # curr = next
    addi x14, x14, -1  # count--
    bne  x14, x0, loop # if count != 0, loop

# === DONE ===
```

```
19      sw   x0, 0(x6)      # clear LEDs
20      jal  x0, halt        # halt (infinite loop)
21
22  # === DISPLAY_AND_DELAY SUBROUTINE ===
23  display:
24      addi x2, x2, -8     # push stack (room for 2 words)
25      sw   x1, 4(x2)      # save return address
26      sw   x10, 0(x2)     # save a0
27      sw   x10, 0(x6)     # write a0 to LEDs
28      addi x13, x0, 5     # delay counter = 5
29  delay:
30      addi x13, x13, -1   # delay--
31      bne  x13, x0, delay # loop until zero
32      lw   x10, 0(x2)     # restore a0
33      lw   x1, 4(x2)      # restore return address
34      addi x2, x2, 8      # pop stack
35      jalr x0, 0(x1)      # return to caller
```

3.5 Stack Usage

The subroutine demonstrates correct RISC-V calling convention:

1. **Prologue:** `sp` is decremented by 8 bytes to make room for two 32-bit words. The return address (`ra`) is saved at `sp+4` and the argument (`a0`) at `sp+0`.
2. **Body:** The subroutine writes to LEDs and runs a delay loop.
3. **Epilogue:** `a0` and `ra` are restored from the stack. `sp` is incremented by 8 to pop the frame. The subroutine returns via `jalr`.

3.6 Simulation Results

The simulation waveform confirms the Fibonacci sequence on the LEDs:

Table 4: Fibonacci Execution Results

Iteration	LED Value
1	1
2	1
3	2
4	3
5	5
6	8
7	13
8	21
9	34
10	55
End	0 (cleared)

(Attach simulation waveform screenshot here)

4 Summary of Modified Files

Table 5: Files Modified and Reason

File	Modification	For Instruction
ALU.v	Added case `4'b0111` for signed comparison	SLT
ALUControl.v	Added funct3 `3'b010` → `4'b0111` mapping	SLT
ALUControl.v	Fixed encoding to match ALU operations	All (bug fix)
MainControl.v	Added `LUI` output signal and opcode case	LUI
TopLevelProcessor.v	Added BGE condition to `branchTaken`	BGE
TopLevelProcessor.v	Added LUI write-back path (imm → rd)	LUI
instructionMemory.v	Updated with test/Fibonacci programs	Part B, C

5 Conclusion

This project successfully demonstrated the integration of RISC-V assembly programs with a hardware-implemented single-cycle processor. The Lab 10 FSM program was executed on the processor from Lab 11, verifying correct instruction sequencing and memory-mapped I/O operation. Three new instructions (SLT, BGE, LUI) of different types were implemented with minimal hardware modifications and verified through simulation. A Fibonacci series program with proper stack-based subroutine calls was designed and executed, producing the correct output sequence on the LEDs. All programs were verified through behavioral simulation and are ready for FPGA demonstration.